
TP 2.1 - GENERADORES PSEUDOALEATORIOS

Manuel Ferraris

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
ferrarismanu@gmail.com

Juan Ignacio Fiele

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
juanfiele2002@gmail.com

María Belén Giannone

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
mariabelengiannone010@gmail.com

María Lucía Munné

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
marilumunne@gmail.com

Agustín Stringa

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
stringaagustin1@gmail.com

June 3, 2026

Keywords Python · Numeros Aleatorios · Generadores Pseudoaleatorios · Simulación

ABSTRACT

1 Introducción

En simulación es fundamental disponer de secuencias de números aleatorios que permitan modelar fenómenos inciertos. Debido a que las computadoras son sistemas determinísticos, generalmente se utilizan generadores de números pseudoaleatorios, los cuales producen secuencias que intentan imitar el comportamiento de números verdaderamente aleatorios.

El objetivo de este trabajo es implementar y analizar distintos generadores de números pseudoaleatorios, comparando su comportamiento mediante pruebas estadísticas de uniformidad e independencia. Además, se incluye una comparación con números aleatorios reales obtenidos mediante la plataforma Random.org.

Para evaluar la calidad de los generadores se aplicaron pruebas estadísticas como Chi-Cuadrado y Rachas, complementadas con representaciones gráficas que permiten identificar patrones y posibles deficiencias en las secuencias generadas.

2 Desarrollo Teórico

2.1 Generadores de números pseudoaleatorios implementados

2.1.1 Generador Congruencial Lineal (GCL)

El generador congruencial lineal es uno de los métodos más utilizados para la generación de números pseudoaleatorios. Su funcionamiento se basa en la relación de recurrencia:

$$[X_{n+1} = (aX_n + c) \bmod m]$$

donde (a) es el multiplicador, (c) es el incremento, (m) es el módulo y (X_0) representa la semilla inicial.

En la implementación realizada se utilizaron los parámetros empleados internamente por Java, permitiendo generar valores pseudoaleatorios normalizados en el intervalo $([0,1))$.

2.1.2 Método de la Parte Media del Cuadrado

Este método fue propuesto por John von Neumann y consiste en elevar al cuadrado una semilla numérica y extraer los dígitos centrales del resultado para formar la nueva semilla.

Aunque históricamente tuvo importancia en los primeros estudios sobre generación de números pseudoaleatorios, presenta limitaciones importantes debido a la aparición de ciclos cortos y convergencia hacia valores repetidos.

2.1.3 Generador RANDU

RANDU es un generador congruencial lineal históricamente conocido por sus deficiencias estadísticas. Utiliza la expresión:

$$[X_{n+1} = (65539X_n) \bmod 2^{31}]$$

A pesar de haber sido ampliamente utilizado en la década de 1960, posteriormente se comprobó que introduce fuertes correlaciones entre valores consecutivos, generando patrones claramente visibles en representaciones gráficas.

2.2 Pruebas de aleatoriedad

Las pruebas de aleatoriedad se aplican a los generadores para evaluar su calidad. Si bien, una prueba simple puede realizarse mediante observación visual de ciertos gráficos, como el histograma o el gráfico de dispersión, no son en absoluto suficientes para validar un generador. Por ello, se aplican pruebas estadísticas formales que evalúan diferentes aspectos de la secuencia generada, como la uniformidad, la independencia y la ausencia de patrones predecibles.

Entre las pruebas estadísticas se pueden diferenciar:

- **empíricas:** Evalúan estadísticas de sucesiones de números.
- **teóricas:** Se establecen las características de las sucesiones usando métodos de teoría de números con base en la regla de recurrencia que generó la sucesión.

Prueba de bondad de ajuste chi cuadrado.

Esta prueba determina si una sola variable categórica se ajusta a una distribución teórica esperada. Las hipótesis de la prueba son:

- H_0 : La variable se ajusta a la distribución teórica F .
- H_1 : La variable no se ajusta a la distribución teórica F .

Procedimiento:

1. Partir el soporte de F en k clases que son exhaustivas y mutuamente
2. Contar el número de observaciones O_i que caen en cada celda i .

Para evaluar la calidad del generador de números pseudoaleatorios en el dominio $[0, 1)$, se realizó una partición del espacio muestral en $k = 10$ sub-intervalos equiprobables de amplitud 0.1. Se establece un nivel de significancia $\alpha = 0.05$ para la prueba, lo que implica que se rechazará la hipótesis nula si el valor calculado del estadístico de prueba excede el valor crítico correspondiente a $k - 1$ (para nuestro caso específico, 9) grados de libertad. La hipótesis nula (H_0) establece que el generador sigue una distribución uniforme continua, lo que implica que la frecuencia esperada E_i para cada intervalo i es constante e igual a n/k , donde n representa el tamaño de la muestra.

De este modo calculamos para cada generador el estadístico siguiendo la fórmula general:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

donde O_i es la frecuencia observada en el intervalo i y E_i es la frecuencia esperada bajo la hipótesis nula.

Resultando en una formula:

$$\chi^2 = \sum_{i=1}^{10} \frac{(O_i - n/10)^2}{n/10}$$

Una vez obtenido el valor del estadístico es posible valorar el mismo comparándolo con el valor crítico de la distribución chi-cuadrado con $k - 1$ grados de libertad, o bien calcular el p-valor asociado a dicho estadístico para determinar si se rechaza o no la hipótesis nula. El p-valor se interpreta como la probabilidad de obtener un valor del estadístico de prueba tan extremo como el observado, bajo la suposición de que la hipótesis nula es verdadera.

Prueba de rachas.

Se define a una racha en una secuencia a "cualquier subsecuencia no vacía de elementos consecutivos del mismo valor." Para nuestro caso partimos de la secuencia de números generados y la transformamos en una secuencia binaria, donde cada elemento se clasifica como "mayor o igual a la media" o "menor a la media".

Las hipótesis de la prueba son:

- H_0 : La secuencia es independiente.
- H_1 : La secuencia no es independiente.

El procedimiento para esta prueba es el siguiente:

1. Transformar la secuencia de números generados en una secuencia binaria.
2. Contar el número total de rachas R en la secuencia binaria.

Bajo la hipótesis nula de independencia, el número de rachas R en una secuencia de longitud n sigue aproximadamente una distribución normal con media μ_R y desviación estándar σ_R dadas por:

$$\mu_R = \frac{2n_1n_2}{n} + 1$$

$$\sigma_R = \sqrt{\frac{2n_1n_2(2n_1n_2 - n_1 - n_2)}{(n_1 + n_2)^2(n_1 + n_2 - 1)}}$$

donde n_1 es el número de elementos en la secuencia binaria que son "mayor o igual a la media" y n_2 es el número de elementos que son "menor a la media". El estadístico de prueba se calcula como:

$$Z = \frac{R - \mu_R}{\sigma_R}$$

donde R es el número total de rachas observadas en la secuencia binaria. Finalmente, se compara el valor absoluto del estadístico Z con el valor crítico de la distribución normal estándar para el nivel de significancia elegido, o se calcula el p-valor asociado al estadístico Z para determinar si se rechaza o no la hipótesis nula de independencia.

En la implementación práctica en *Python* de esta prueba, se utilizó la función `runstest_1samp` del módulo `statsmodels.sandbox.stats.runs`, que realiza automáticamente los cálculos necesarios para obtener el estadístico de prueba y el p-valor correspondiente.

2.3 Generadores de números aleatorios reales implementados

Para la implementación de generadores de números aleatorios reales se desarrolló una función mediante integración con la API de Random.org, que proporciona números aleatorios generados a partir de fenómenos físicos, como el ruido atmosférico.

La API pública "gratuita" permite obtener números aleatorios enteros en el rango $[1, 10]$, y se pueden solicitar secuencias de números con una longitud específica. Por este motivo fue necesario adaptar las pruebas y gráficas a comportamientos de variables aleatorias discretas, utilizando intervalos de clase adecuados para la prueba de chi-cuadrado y ajustando el gráfico de barras para mostrar la frecuencia de cada número entero generado.

3 Resultados

Se ejecutaron las pruebas estadísticas sobre los distintos generadores implementados utilizando muestras de tamaño $n = 10000$.

Para cada generador se obtuvo:

- Histograma de frecuencias.
- Bitmap de distribución.
- Estadístico Chi-Cuadrado.
- Prueba de Rachas.

4 Discusión y Conclusiones

Referencias

- [1] Cátedra de Simulación. (2026). *Enunciado Trabajo Práctico 2.1: GENERADORES PSEUDOALEATORIOS*. Universidad Tecnológica Nacional - Facultad Regional Rosario (UTN-FRRO).
- [2] Cátedra de Probabilidad y Estadística. *Tratamiento de Datos*. Universidad Tecnológica Nacional - Facultad Regional Rosario (UTN-FRRO). [2024].
- [3] Wikipedia. *Ruleta*. Recuperado de: <https://es.wikipedia.org/wiki/Ruleta>.
- [4] Cornell University. *arXiv Template for Preprints*.
- [5] Matplotlib Development Team. (2024). *Matplotlib API Reference Documentation*. Recuperado de: <https://matplotlib.org/stable/api/index.html>
- [6] NumPy Developers. (2024). *NumPy Reference Documentation*. Recuperado de: <https://numpy.org/doc/stable/reference/index.html>
- [7] Python Software Foundation. *Source code for random.py*. GitHub - CPython Repository. Recuperado de: <https://github.com/python/cpython/blob/main/Lib/random.py>
- [8] SciPy Developers. (2024). *scipy.stats.chisquare Documentation*. Recuperado de: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chisquare.html>
- [9] Statsmodels Developers. (2024). *statsmodels.sandbox.stats.runs.runstest_1samp Documentation Documentation*. Recuperado de: https://www.statsmodels.org/stable/generated/statsmodels.sandbox.stats.runs.runstest_1samp.html
- [10] Random.org. (2024). *Random.org HTTP API Documentation*. Recuperado de: <https://www.random.org/clients/http/api/>